

Lista Teórica E2 — Gabarito

Curso de Aprendizado de Máquina — UFF

Prof. Gabriel Sanfins

Aula E2 — Redução de Dimensionalidade (PCA e t-SNE)

Exercício 1 — PCA na mão. Quatro pontos em $\mathbb{R}^2$:

$$(3, 1), \quad (1, 3), \quad (-3, -1), \quad (-1, -3).$$

- (a) Verifique que a média é a origem e calcule a matriz de covariância amostral $\mathbf{S} = \frac{1}{n} \mathbf{X}^\top \mathbf{X}$.
- (b) Encontre os autovalores e autovetores de $\mathbf{S}$. Escreva as duas componentes principais como vetores unitários.
- (c) Qual a proporção da variância explicada pela primeira componente?
- (d) Calcule as coordenadas dos quatro pontos no novo sistema (isto é, projete cada um sobre as duas componentes). Que estrutura geométrica isso revela?

Solução

(a) A soma das coordenadas é $(3 + 1 - 3 - 1, 1 + 3 - 1 - 3) = (0, 0)$, logo a média é a origem e os dados já estão centrados. Então

$$\mathbf{X}^\top \mathbf{X} = \begin{pmatrix} 9 + 1 + 9 + 1 & 3 + 3 + 3 + 3 \\ 3 + 3 + 3 + 3 & 1 + 9 + 1 + 9 \end{pmatrix} = \begin{pmatrix} 20 & 12 \\ 12 & 20 \end{pmatrix}, \quad \mathbf{S} = \frac{1}{4} \mathbf{X}^\top \mathbf{X} = \begin{pmatrix} 5 & 3 \\ 3 & 5 \end{pmatrix}.$$

(b) Para uma matriz da forma $\begin{pmatrix} a & b \\ b & a \end{pmatrix}$ os autovetores são sempre $(1, 1)$ e $(1, -1)$, com autovalores $a + b$ e $a - b$. Conferindo:

$$\mathbf{S} \begin{pmatrix} 1 \\ 1 \end{pmatrix} = \begin{pmatrix} 8 \\ 8 \end{pmatrix} = 8 \begin{pmatrix} 1 \\ 1 \end{pmatrix}, \quad \mathbf{S} \begin{pmatrix} 1 \\ -1 \end{pmatrix} = \begin{pmatrix} 2 \\ -2 \end{pmatrix} = 2 \begin{pmatrix} 1 \\ -1 \end{pmatrix}.$$

Logo $\lambda_1 = 8$ e $\lambda_2 = 2$, com componentes principais

$$\mathbf{v}_1 = \frac{1}{\sqrt{2}}(1, 1) \approx (0,7071; 0,7071), \quad \mathbf{v}_2 = \frac{1}{\sqrt{2}}(1, -1) \approx (0,7071; -0,7071).$$

(Sinal e ordem: os autovetores são definidos a menos de sinal, e a convenção é ordená-los por autovalor decrescente.)

(c) $\frac{\lambda_1}{\lambda_1 + \lambda_2} = \frac{8}{10} = 80\%$.

(d) Projetando, $z_j = \mathbf{x}^\top \mathbf{v}_j$:

ponto	z_1 (sobre $\mathbf{v}_1$)	z_2 (sobre $\mathbf{v}_2$)
$(3, 1)$	$4/\sqrt{2} = 2\sqrt{2} \approx 2,8284$	$2/\sqrt{2} = \sqrt{2} \approx 1,4142$
$(1, 3)$	$2\sqrt{2}$	$-\sqrt{2}$
$(-3, -1)$	$-2\sqrt{2}$	$-\sqrt{2}$
$(-1, -3)$	$-2\sqrt{2}$	$\sqrt{2}$

A estrutura fica evidente: os quatro pontos formam um **retângulo** centrado na origem, com o lado maior ($2 \times 2\sqrt{2} = 5,66$) na direção $(1, 1)$ e o menor ($2 \times \sqrt{2} = 2,83$) na direção

$(1, -1)$. No sistema original isso estava escondido, porque o retângulo está a 45° ; a PCA apenas girou os eixos para alinhá-los aos lados.

É a leitura geométrica da PCA em uma frase: *ela procura o sistema de coordenadas em que a nuvem tem eixos alinhados*, e a variância ao longo de cada novo eixo é o autovalor correspondente. Aqui a razão entre os lados é $2\sqrt{2}/\sqrt{2} = 2$, e a razão entre os desvios-padrão é $\sqrt{8}/\sqrt{2} = 2$ — consistente.

Exercício 2 — quantas componentes guardar. Uma matriz de dados com $d = 8$ covariáveis tem autovalores da matriz de covariância

$$\lambda = (12,5; 6,2; 3,1; 1,4; 0,9; 0,5; 0,3; 0,1).$$

- Calcule a proporção de variância explicada por cada componente e a proporção acumulada.
- Quantas componentes são necessárias para reter ao menos 90% da variância? E 99%?
- O que significa `PCA(n_components=0.90)` no `scikit-learn`, e por que é uma forma mais conveniente de especificar a redução do que dar o número de componentes?
- A soma dos autovalores vale 25. A que quantidade dos dados originais esse número corresponde? O que isso diz sobre a necessidade de **padronizar** antes da PCA?

Solução

(a) A soma é $\sum_j \lambda_j = 25$.

componente	1	2	3	4	5	6	7	8
λ_j	12,5	6,2	3,1	1,4	0,9	0,5	0,3	0,1
proporção	0,500	0,248	0,124	0,056	0,036	0,020	0,012	0,004
acumulada	0,500	0,748	0,872	0,928	0,964	0,984	0,996	1,000

(b) Para 90%: três componentes dão 87,2%, ainda insuficiente; com quatro chega-se a 92,8%. São necessárias **4 componentes**. Para 99%: sete componentes dão 99,6% e seis dão 98,4%; são necessárias **7**.

Vale reparar no rendimento decrescente: as 4 primeiras componentes carregam 92,8%, e as 4 últimas somam 7,2%. Guardar metade das colunas custa menos de 8% da variância.

(c) Quando `n_components` é um *float* em $(0,1)$, o `scikit-learn` o interpreta como a **proporção de variância a reter**, e escolhe automaticamente o menor número de componentes que a atinge. Neste exemplo, `PCA(n_components=0.90)` guardaria 4.

É mais conveniente porque 0,90 é uma quantidade **interpretável e transferível**: significa a mesma coisa em qualquer conjunto de dados, enquanto “4 componentes” depende de d e da estrutura de correlação. Num **Pipeline** com validação cruzada, o número escolhido pode até variar entre dobras — e é isso que se quer, já que o critério é o mesmo.

(d) A soma dos autovalores é o **traço** de $\mathbf{S}$, isto é, a soma das variâncias das d covariáveis originais — a “variância total” dos dados. (Girar os eixos não muda o traço.)

E aí está o problema: essa soma é medida nas **unidades** das covariáveis. Se uma delas estiver em milímetros e outra em quilômetros, a primeira terá variância 10^{12} vezes maior e sozinha responderá por praticamente toda a variância total. A primeira componente principal será, essencialmente, essa covariável — não porque ela importa, mas porque a régua dela é menor.

Por isso, salvo quando todas as covariáveis já estão na mesma unidade e a escala relativa entre elas é significativa, **padroniza-se antes da PCA** — o que equivale a fazer a decomposição da matriz de *correlação* em vez da de covariância. É o mesmo argumento do Exercício 2(c) da Lista Teórica 02, sobre Ridge e Lasso.

Exercício 3 — o que o t-SNE preserva, e o que não preserva. O t-SNE constrói, no espaço original, probabilidades p_{ij} de que i escolha j como vizinho, e procura pontos z_i em $\mathbb{R}^2$ cujas probabilidades q_{ij} minimizem $KL(p \| q)$.

- (a) A divergência de Kullback–Leibler $\sum_{ij} p_{ij} \log(p_{ij}/q_{ij})$ não é simétrica. Que tipo de erro ela pune muito, e que tipo ela quase não pune? *Sugestão:* olhe o que acontece com o somando quando p_{ij} é grande e q_{ij} pequeno, e vice-versa.
- (b) Conclua o que se pode e o que não se pode ler num diagrama de t-SNE sobre: (i) pontos que aparecem próximos; (ii) a distância entre dois grupos distantes; (iii) o tamanho relativo de dois grupos.
- (c) O parâmetro **perplexity** controla, grosso modo, quantos vizinhos cada ponto considera. O que se espera do diagrama com **perplexity** muito pequena? E muito grande?
- (d) Por que não se usa t-SNE como etapa de um Pipeline de predição, do jeito que se usa PCA?

Solução

(a) O somando é $p_{ij} \log(p_{ij}/q_{ij})$, e o peso de cada termo é p_{ij} — a probabilidade no espaço *original*.

Se p_{ij} é grande (os pontos são vizinhos de verdade) e q_{ij} é pequeno (ficaram longe no mapa), o logaritmo vai a $+\infty$ e o termo é multiplicado por um p_{ij} grande: **punição enorme**. Separar vizinhos verdadeiros é caríssimo.

Se p_{ij} é pequeno (os pontos são distantes) e q_{ij} é grande (ficaram perto no mapa), o logaritmo vai a $-\infty$, mas o termo é multiplicado por $p_{ij} \approx 0$: **punição quase nula**. Aproximar pontos distantes é de graça.

Em uma frase: a KL nesta direção pune *omitir* vizinhança verdadeira e é quase indiferente a *inventar* vizinhança falsa entre pontos distantes.

(b)

(i) **Pontos próximos no mapa: pode confiar.** Como separar vizinhos verdadeiros é caríssimo, dois pontos que aparecem juntos quase sempre eram vizinhos no espaço original. É exatamente o que o t-SNE otimiza.

(ii) **Distância entre grupos distantes: não significa nada.** Se dois grupos estão longe no espaço original, todos os p_{ij} entre eles são praticamente zero, e a KL é indiferente a onde eles vão parar no mapa. Dizer “o grupo A está mais perto do B do que do C” a partir de um diagrama de t-SNE é uma leitura sem base.

(iii) **Tamanho relativo dos grupos: também não.** O t-SNE normaliza a escala local de cada ponto pela **perplexity**, de modo que uma nuvem densa e uma esparsa podem ocupar áreas parecidas no mapa. A área de um agrupamento no diagrama não é proporcional a nada.

(c) Com **perplexity** muito pequena, cada ponto só considera pouquíssimos vizinhos: o mapa se fragmenta em muitos grupinhos minúsculos, e a estrutura de médio alcance

desaparece. Com **perplexity** muito grande, cada ponto considera quase todo mundo, os p_{ij} ficam quase uniformes e o mapa tende a uma nuvem única e sem estrutura — perto do que o PCA daria.

Vale reter que **perplexity** não é um parâmetro que se “otimiza”: não há função de risco a minimizar. Rodar com dois ou três valores e ver o que *sobrevive* aos três é a conduta usual.

(d) Por duas razões, e a primeira é decisiva.

Não existe transform. O t-SNE não aprende um mapa $\mathbb{R}^d \rightarrow \mathbb{R}^2$; ele resolve um problema de otimização cujas variáveis são as *posições dos pontos que você lhe deu*. Não há como projetar uma observação nova sem refazer o ajuste — e refazê-lo com o ponto novo muda as posições de todos os outros. Um ‘Pipeline’ precisa de um passo que possa ser ajustado no treino e aplicado à validação, e o t-SNE não oferece isso. (Na API do **scikit-learn** isso aparece como um **fit_transform** sem **transform** correspondente.)

E o objetivo é outro. O t-SNE existe para produzir uma figura que um humano olha. As duas coordenadas que ele devolve não são features com significado — não são combinações lineares interpretáveis das originais, como as da PCA, e nem sequer são determinadas de forma única (rodar de novo com outra semente dá outro mapa). Alimentar um classificador com elas é usar como covariável o resultado de um algoritmo de visualização.

A PCA, ao contrário, é uma transformação linear fixa: ela aprende uma matriz no treino e a aplica a qualquer ponto novo. É por isso que ela cabe num **Pipeline** e o t-SNE não.

Exercício 4 ★ — PCA e SVD são a mesma coisa. Seja $\mathbf{X}$ a matriz $n \times d$ dos dados **centrados**, com decomposição em valores singulares $\mathbf{X} = \mathbf{U}\mathbf{D}\mathbf{V}^\top$, em que $\mathbf{U}$ é $n \times r$ com colunas ortonormais, $\mathbf{D}$ é diagonal $r \times r$ com $d_1 \geq d_2 \geq \dots \geq d_r > 0$, e $\mathbf{V}$ é $d \times r$ com colunas ortonormais.

- Mostre que as colunas de $\mathbf{V}$ são autovetores de $\mathbf{S} = \frac{1}{n}\mathbf{X}^\top\mathbf{X}$ e que o autovalor associado a $\mathbf{v}_j$ é $\lambda_j = d_j^2/n$.
- Conclua que as componentes principais podem ser calculadas por SVD, sem formar $\mathbf{X}^\top\mathbf{X}$. Por que isso importa numericamente?
- As coordenadas dos dados nas k primeiras componentes são as k primeiras colunas de $\mathbf{UD}$. Verifique isso a partir de $\mathbf{Z} = \mathbf{XV}$.
- O teorema de Eckart–Young diz que $\mathbf{X}_k = \mathbf{U}_k\mathbf{D}_k\mathbf{V}_k^\top$ é a melhor aproximação de posto k de $\mathbf{X}$, no sentido de minimizar $\|\mathbf{X} - \mathbf{A}\|_F$ entre todas as $\mathbf{A}$ de posto $\leq k$. Que interpretação isso dá à PCA?

Solução

(a) Usando $\mathbf{U}^\top\mathbf{U} = \mathbf{I}$,

$$\mathbf{X}^\top\mathbf{X} = (\mathbf{UDV}^\top)^\top(\mathbf{UDV}^\top) = \mathbf{VD} \underbrace{\mathbf{U}^\top\mathbf{U}}_{\mathbf{I}} \mathbf{DV}^\top = \mathbf{VD}^2\mathbf{V}^\top.$$

Multiplicando à direita por $\mathbf{v}_j$ (a j -ésima coluna de $\mathbf{V}$) e usando $\mathbf{V}^\top\mathbf{v}_j = \mathbf{e}_j$:

$$\mathbf{X}^\top\mathbf{X}\mathbf{v}_j = \mathbf{VD}^2\mathbf{e}_j = d_j^2\mathbf{v}_j.$$

Dividindo por n , $\mathbf{S}\mathbf{v}_j = (d_j^2/n)\mathbf{v}_j$: os $\mathbf{v}_j$ são autovetores de $\mathbf{S}$ com autovalores $\lambda_j = d_j^2/n$. Como $d_1 \geq \dots \geq d_r$, eles já vêm em ordem decrescente de autovalor.

(b) Basta calcular a SVD de $\mathbf{X}$ e ler $\mathbf{V}$ e $\mathbf{D}$; nunca é preciso formar $\mathbf{X}^\top\mathbf{X}$.

Isso importa porque formar $\mathbf{X}^\top \mathbf{X}$ eleva ao quadrado o número de condição da matriz. Se $\mathbf{X}$ tem valores singulares indo de d_1 a d_r , o número de condição de $\mathbf{X}$ é d_1/d_r e o de $\mathbf{X}^\top \mathbf{X}$ é $(d_1/d_r)^2$. Em precisão dupla, com $\kappa(\mathbf{X}) \approx 10^8$ — nada absurdo em dados reais correlacionados — o produto tem $\kappa \approx 10^{16}$ e as componentes menores viram ruído numérico.

É o mesmo motivo pelo qual bons solucionadores de mínimos quadrados usam QR ou SVD em vez de resolver as equações normais diretamente (Aula 02).

(c) As coordenadas nas componentes principais são as projeções sobre os $\mathbf{v}_j$, isto é $\mathbf{Z} = \mathbf{X}\mathbf{V}$. Substituindo a SVD e usando $\mathbf{V}^\top \mathbf{V} = \mathbf{I}$:

$$\mathbf{Z} = \mathbf{X}\mathbf{V} = \mathbf{U}\mathbf{D}\mathbf{V}^\top \mathbf{V} = \mathbf{U}\mathbf{D}.$$

As k primeiras colunas de $\mathbf{Z}$ são, portanto, as k primeiras de $\mathbf{U}\mathbf{D}$ — ou seja, $\mathbf{u}_j d_j$ para $j = 1, \dots, k$. (Confere com o item (a): a variância da coluna j é $\frac{1}{n} \|\mathbf{u}_j d_j\|^2 = d_j^2/n = \lambda_j$, já que $\|\mathbf{u}_j\| = 1$.)

(d) Dá à PCA uma caracterização que não menciona variância nenhuma: **a projeção sobre as k primeiras componentes principais é a aproximação de posto k que menos distorce a matriz de dados**, medida pela soma dos quadrados de todos os erros de reconstrução.

São duas leituras equivalentes do mesmo objeto:

- *maximizar* a variância retida nas k direções escolhidas;
- *minimizar* o erro quadrático de reconstrução ao voltar de k dimensões para d .

E elas são equivalentes pelo mesmo mecanismo do Exercício 4 da Lista Teórica E1: a soma de quadrados total é fixa, então maximizar a parte “explicada” é minimizar a parte “residual”. O k -médias parte a soma de quadrados entre grupos e dentro de grupos; a PCA a parte entre direções retidas e direções descartadas. É a mesma identidade, aplicada a dois tipos de estrutura.

Os exercícios marcados com $\star$ são opcionais e vão além do que a prova cobra.